Faculdade: FIAP - Faculdade de Informática e Administração Paulista

Curso: Inteligência Artificial (Graduação)

Turma: 1TIAOR - 2024/2

Integrantes:

- RM560173 - Gabriel de Oliveira Soares Ribeiro
- RM559926 - Marcos de Souza Trazzini
- RM559800 - Jonas Felipe dos Santos Lima
- RM559645 - Edimilson Ribeiro da Silva

Link do projeto no GitHub: https://github.com/ribeirogab/youvisa

Vídeo demonstrativo: https://youtu.be/621cVSKsvQU

O código fonte e recursos do projeto estão organizados na pasta `./app`. A Sprint 4 substituiu n8n + Lambdas Python + AWS por um pipeline multi-agente em TypeScript rodando localmente via Docker Compose.

Arquivos principais

- `./app/agent/src/server.ts` - Servidor Hono que recebe webhook do Telegram diretamente (substitui o n8n)
- `./app/agent/src/orchestrator/pipeline.ts` - Orquestrador que executa os 6 steps do pipeline multi-agente
- `./app/agent/src/orchestrator/tracer.ts` - Captura `{step, started_at, duration_ms, output}` para auditoria por step
- `./app/agent/src/agents/input-filter.ts` - Bloqueio determinístico de prompt injection (regex pré-LLM, ~5ms)
- `./app/agent/src/agents/intent-classifier.ts` - Classificador de intent com Claude Haiku 4.5 (JSON schema)
- `./app/agent/src/agents/entity-extractor.ts` - Extração de entidades (visa_type, country, doc_type, email)
- `./app/agent/src/agents/lookup.ts` - Lookup determinístico no MongoDB (status, processos, documentos)
- `./app/agent/src/agents/response-generator.ts` - Gerador de resposta com 8 few-shot + 10 regras de governança
- `./app/agent/src/agents/output-filter.ts` - Filtro de saída (rejeita prazos, códigos internos, placeholders)
- `./app/agent/src/classifier/document-flow.ts` - Validação + upload + classificação de documentos
- `./app/agent/src/classifier/claude-vision.ts` - Classificação multimodal via Anthropic Messages API
- `./app/agent/src/auth/jwt.ts` - Emissão e verificação de JWT HS256 para o Portal do Cliente (TTL 24h)
- `./app/validation/src/validator.py` - Validação de qualidade visual (variance-of-Laplacian, FastAPI + OpenCV)
- `./app/api/src/routes.ts` - Rotas da API (users, conversations, processes, files, interactions)
- `./app/api/src/controllers/interaction.controller.ts` - Persistência e leitura dos `interaction_logs`

- `./app/frontend/src/app/dashboard/interactions/page.tsx` - Console com `agent_trace` expansível por step
- `./app/frontend/src/app/portal/[token]/page.tsx` - Portal do Cliente autenticado por JWT
- `./scripts/record-demo-v2-tg.mjs` - Gravação automatizada do vídeo demonstrativo via Playwright

Diretórios importantes

- `./app/agent/` - Serviço de agentes (Hono + Claude Agent SDK, port 7777)
- `./app/api/` - Backend API (TypeScript/Fastify, port 5555)
- `./app/validation/` - Serviço de validação de imagens (Python/FastAPI, port 8001)
- `./app/frontend/` - Console do Operador + Portal do Cliente (Next.js 16, port 3010)
- `./context/specs/2026-04-26-sprint-4-multi-agent/` - Especificação técnica da Sprint 4
- `./context/learnings/` - Notas atômicas com gotchas e decisões arquiteturais
- `./scripts/` - Gravação da demo, smoke tests e setup do ambiente local
- `./docker-compose.yml` - Stack local (mongo, minio, validation, agent, api, frontend)

Principais recursos

- **Pipeline Multi-Agente Tipado** - 6 steps independentes (input-filter → intent → entity → lookup → response → output-filter), cada um testável e debugável separadamente, com saída validada por Zod
- **Auditoria Completa por Step (`agent_trace`)** - Toda mensagem produz um `interaction_log` com latência por step; visível tanto para o operador quanto para o cliente
- **Defesa em Profundidade contra Prompt Injection** - Input filter determinístico bloqueia padrões conhecidos antes de qualquer LLM, em ~5ms, sem gastar token
- **Portal do Cliente Autenticado por JWT** - Cliente pede `abrir portal` no Telegram, recebe link com JWT HS256 (TTL 24h) que dá acesso ao próprio status, timeline, documentos e histórico de interações
- **Document Upload End-to-End** - Validação de qualidade (OpenCV, variance-of-Laplacian) → upload para MinIO → classificação por Claude Vision (Passaporte / RG / Comprovante / Formulário)
- **Console do Operador Renovado** - Visão unificada de conversas multi-canal, processos, interações com `agent_trace`, documentos classificados e usuários
- **Notificações Determinísticas** - Mudança de status pelo operador dispara mensagem no Telegram com template fixo, sem passar pelo LLM (zero risco de alucinação)
- **Handoff Bot ↔ Humano sem Regressão** - Conversa em status `transferred` silencia o bot integralmente; operador devolve com um clique e bot retoma
- **Stack 100% Local via Docker Compose** - Mongo, MinIO, validation, agent, api e frontend rodam em containers; zero dependência de AWS na Sprint 4
- **Autenticação Claude SDK via Volume Docker** - Token OAuth do plano Max é montado em `claude_home`, sincronizado do Keychain do macOS quando expira
- **Governança de IA** - System prompts incluem 10 regras explícitas (sem prazos, sem decisões institucionais, só dados reais, rótulos amigáveis); `output-filter` rejeita resposta que viole as regras
- **Demo Reproduzível** - `make record` orquestra ngrok + webhook + Playwright e gera `docs/demo-sprint-4.mp4` com captions, spotlights e zoom-ins de ~5:45

Tecnologias utilizadas

Categoria	Tecnologia

Backend API	TypeScript, Fastify, tsyringe (DI)
Serviço de Agentes	TypeScript, Hono, Claude Agent SDK
Validação de Imagens	Python, FastAPI, OpenCV
Frontend	Next.js 16, React 19, Tailwind CSS, shadcn/ui
IA/ML — Texto	Claude Haiku 4.5 (via Claude Agent SDK)
IA/ML — Visão	Claude (Anthropic Messages API direto, multimodal)
Banco de Dados	MongoDB (local, container)
Armazenamento	MinIO (S3-compatible, local)
Auth do Portal	JWT HS256 (24h TTL)
Mensageria	Telegram Bot API (webhook via ngrok)
Orquestração Local	Docker Compose, GNU Make
Demo	Playwright (Chromium headed + recordVideo), ffmpeg